Guía de Actividades **Práctico- Experimentales** Nro. 001

1. Datos Generales

Asignatura	Sistemas Digitales
Ciclo	5to
Unidad	Uno
Resultado de aprendizaje de la unidad	R1. Diseña arquitectura de microcontroladores, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	1
Título de la Práctica	Uso de Wokwi y Aruino IDE en Microcontroladores AVR
Nombre del Docente	José O. Guamán Q.
Fecha	09 de abril de 2026
Horario	11h30 a 13h30
Lugar	Aula
Tiempo planificado en el Sílabo	2

2. Objetivo(s) de la Práctica:

- Comprender y aplicar el entorno de desarrollo Wokwi para simulación de circuitos con microcontroladores AVR.
- Familiarizarse con la estructura basica de un programa en Arduino IDE, identificando las funciones setup() y loop(), declaracion de variables, tipos de datos, constantes y operaciones bitwise.

3. Materiales y reactivos:

- 1 Protoboard
- 1 Arduino UNO
- 1 LED Rojo 5mm
- 1 Resistencia 220 ohmios
- 1 Cable USB
- Cables puente

4. Equipos y herramientas

- Computadora
- Arduino IDE
- Wokwi (wokwi.com)

5. Procedimiento / Metodología

Paso 1: Ingresar a Wokwi y crear un proyecto Arduino UNO. Explorar la interfaz del simulador.

Paso 2: Montar circuito básico en protoboard virtual: LED en pin D13 con resistencia 220 ohmios a GND.

Paso 3: Escribir programa que declare variables de diferentes tipos (int, byte, long, float, bool), realice operaciones bitwise (AND, OR, XOR, NOT, shift) y las aplique para controlar el LED.

Paso 4: Verificar funcionamiento en simulador Wokwi observando el LED.

Paso 5: Replicar en Microchip Studio y comparar diferencias en estructura y compilación.

Paso 6: Documentar resultados y análisis comparativo entre entornos

Algoritmo

```
ALGORITMO APE 01 - Operaciones Bitwise INICIO
  LED_PIN <-- 13
  estadoLed <-- 0
  contador <-- 0
  CONFIGURAR LED_PIN como SALIDA
  INICIAR serial 9600
  IMPRIMIR resultados de AND, OR, XOR, NOT, Shift estadoLed
  <-- estadoLed OR (1 << 0)
  REPETIR
    estadoLed <-- estadoLed XOR 0b00000001
    SI (estadoLed AND 1) = 1 ENTONCES ENCENDER LED
    SINO APAGAR LED
    ESPERAR 500ms
    contador <-- (contador + 1) MOD 8 IMPRIMIR
    (1 << contador)
    ESPERAR 500ms
  FIN REPETIR
FIN
```

6. Resultados esperados:

El LED en D13 parpadea cada 500ms por la operacion XOR. En el monitor serie se visualizan los resultados de las operaciones bitwise en binario, demostrando como afectan los bits al estado de los pines

Se debe entregar el diagrama de conexión, el código realizado y el funcionamiento

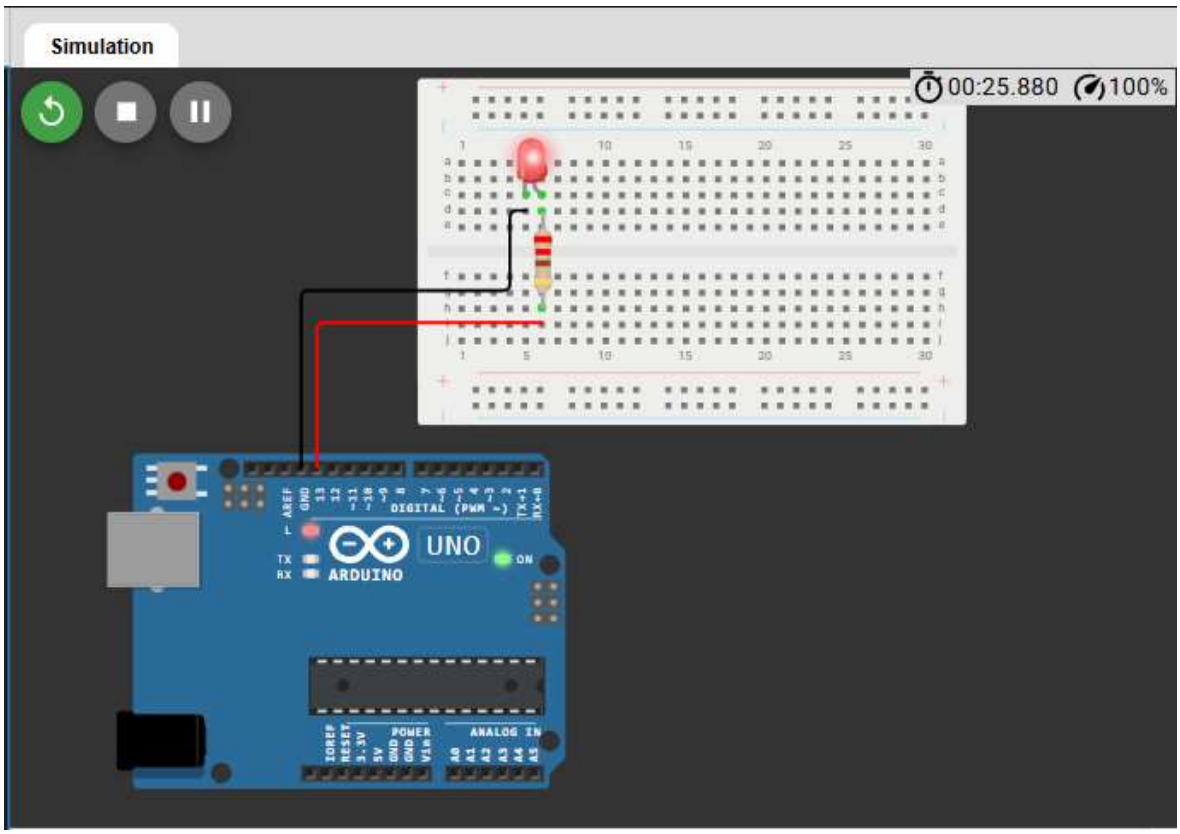
- **Simulador Wokwi**

Código

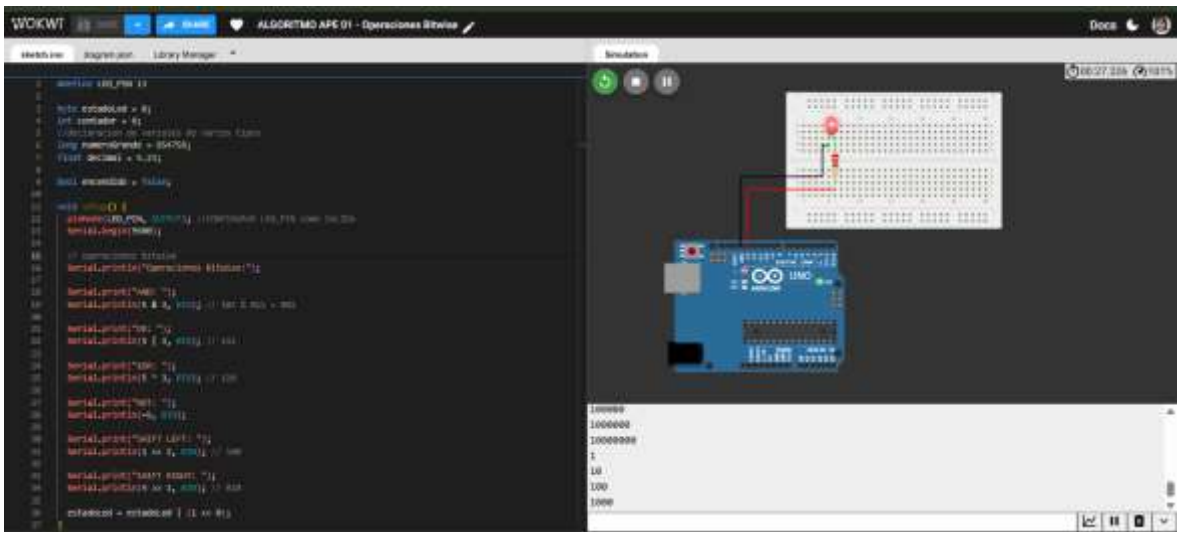
sketch.ino | diagram.json | Library Manager

```
#define LED_PIN 13

byte estadoLed = 0;
int contador = 0;
//declaracion de variales de varios tipos
long numeroGrande = 854758;
float decimal = 5.23;

bool encendido = false;

void setup() {
  pinMode(LED_PIN, OUTPUT); //CONFIGURAR LED_PIN como SALIDA
  Serial.begin(9600);

  // operaciones bitwise
  Serial.println("Operaciones Bitwise:");

  Serial.print("AND: ");
  Serial.println(5 & 3, BIN); // 101 & 011 = 001

  Serial.print("OR: ");
  Serial.println(5 | 3, BIN); // 111

  Serial.print("XOR: ");
  Serial.println(5 ^ 3, BIN); // 110

  Serial.print("NOT: ");
  Serial.println(~5, BIN);

  Serial.print("SHIFT LEFT: ");
  Serial.println(1 << 2, BIN); // 100

  Serial.print("SHIFT RIGHT: ");
  Serial.println(4 >> 1, BIN); // 010

  estadoLed = estadoLed | (1 << 0);
}

void loop() {

  // Cambiar estado con XOR
  estadoLed = estadoLed ^ 0b00000001;

  // Verificar estado
  if ((estadoLed & 1) == 1) {
    digitalWrite(LED_PIN, HIGH);
  } else {
    digitalWrite(LED_PIN, LOW);
  }

  delay(500);

  contador = (contador + 1) % 8;

  //Serial.print("contador: ");
  Serial.println(1 << contador, BIN);

  delay(500);
}
```

Simulación




```
Operaciones Bitwise:
AND: 1
OR: 111
XOR: 110
NOT: 11111111111111111111111111111010
SHIFT LEFT: 100
SHIFT RIGHT: 10
10
100
1000
10000
100000
1000000
10000000
1
10
100
1000
10000
100000
1000000
10000000
```



Link: https://wokwi.com/projects/460842123873084417

- **Simulador Arduino IDE**

File Edit Sketch Tools Help

Arduino UNO

sensorPeso.ino

```
#define LED_PIN 13

byte estadoLed = 0;
int contador = 0;
//declaracion de variales de varios tipos
long numeroGrande = 854758;
float decimal = 5.23;

bool encendido = false;

void setup() {
  pinMode(LED_PIN, OUTPUT); //CONFIGURAR LED_PIN como SALIDA
  Serial.begin(9600);

  // operaciones bitwise
  Serial.println("Operaciones Bitwise:");

  Serial.print("AND: ");
  Serial.println(5 & 3, BIN); // 101 & 011 = 001

  Serial.print("OR: ");
  Serial.println(5 | 3, BIN); // 111

  Serial.print("XOR: ");
  Serial.println(5 ^ 3, BIN); // 110

  Serial.print("NOT: ");
  Serial.println(~5, BIN);

  Serial.print("SHIFT LEFT: ");
  Serial.println(1 << 2, BIN); // 100
```

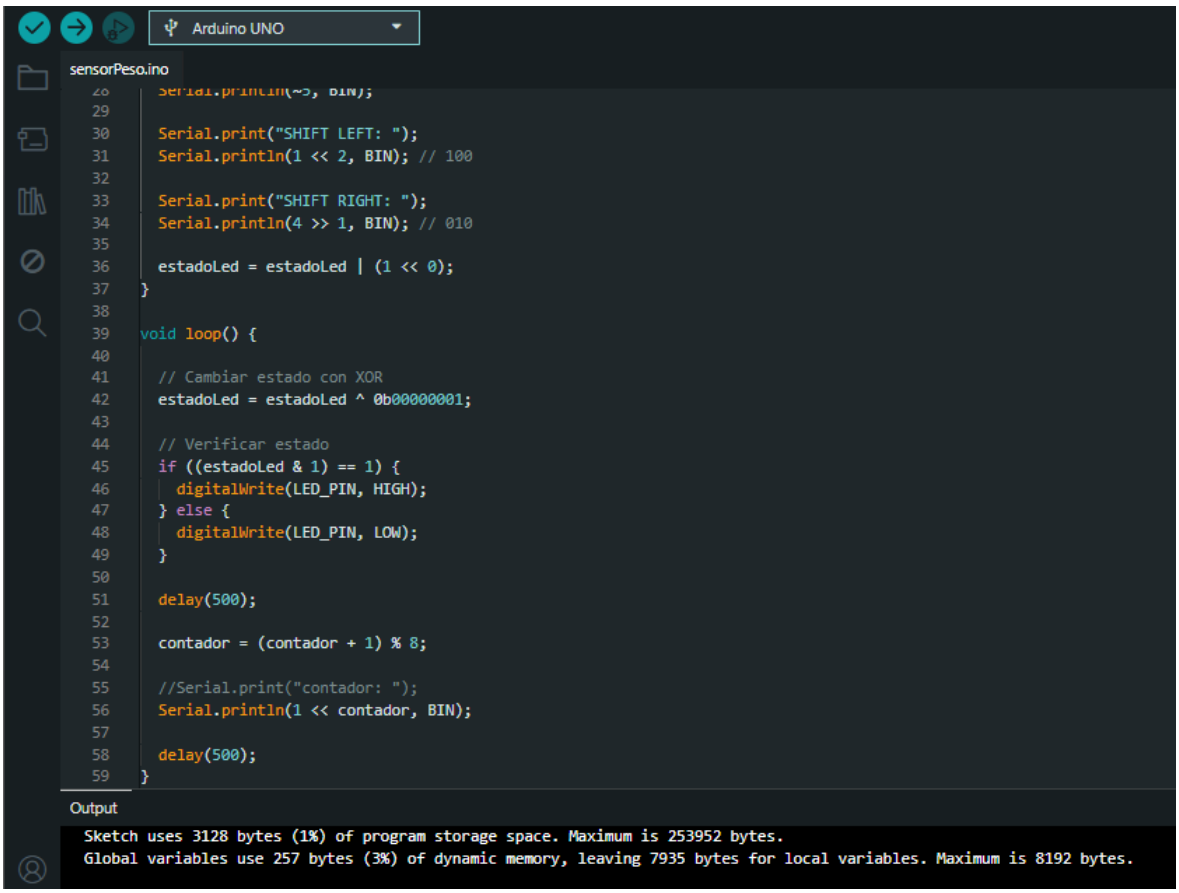


Tabla comparativa

Wokwi	IDE Arduino
Simulador en línea	Entorno en línea y para descargable
Permite programar código para el respectivo funcionamiento de los componentes seleccionados a simular,	Se puede programar el código, pero a diferencia de Wokwi , este requiere componentes en físico para comprobar el si el código es funcional.
Se visualiza el circuito y el funcionamiento del mismo.	No se puede visualizar, ayuda cargar código directamente al hardware.
No se necesita componentes físicos	Se requiere componentes en físicos.
Es de fácil uso y sirve para realizar pruebas.	Realizar pruebas reales y verificar el comportamiento del circuito en la realidad.

7. Preguntas de Control:

1. **Explique que es una operacion bitwise y su importancia en microcontroladores.**

Una operación bitwise (a nivel de bits) es aquellas que trabaja directamente sobre la representación binaria de un número, bit a bit, las operaciones mas comunes son AND, OR, XOR, NOT.

Los microcontroladores como el ATmega328 (Arduino) controlan hardware mediante registros de 8 bits, donde cada bit representa un pin, una bandera de estado o una configuración. Las operaciones bitwise permiten modificar un bit específico sin alterar los demás importante

para controlar pines, habilitar interrupciones, configurar periféricos (UART, SPI, I2C, PWM) y leer flags de estado, todo esto en instrucciones de un solo ciclo de reloj.

2. **Diferencias entre tipos byte, int, long, float en Arduino con ejemplos.**

Tipo	Tamaño	Rango	Uso típico
byte	8 bits	0 – 255	Datos de registro, colores
int	16 bits	-32,768 – 32,767	Contadores, sensores
long	32 bits	±2,147,483,647	millis(), distancias
float	32 bits	$\pm 3.4 \times 10^{38}$ (6-7 decimales)	Sensores analógicos, cálculos

Ejemplos:

byte ledPin => 13; *solo necesita 0-255*

int temp => -20; *puede ser negativo*

long tiempo => millis(); *millis() retorna long*

float voltaje => analogRead(A0) * (5.0 / 1023.0); *necesita decimales*

3. **Resultado de 0b11001100 & 0b10101010 mostrando procedimiento.**

```
        1 1 0 0 1 1 0 0
(&) *   1 0 1 0 1 0 1 0
    ———————————————————
        1 0 0 0 1 0 0 0

        0b10001000
```

4. **Uso del desplazamiento de bits en control de perifericos.**

El desplazamiento (<< y >>) mueve todos los bits hacia la izquierda o derecha, lo que equivale a multiplicar o dividir por potencias de 2, y es la técnica estándar para direccionar bits en registros.

5. **Ventajas y desventajas de Wokwi vs hardware fisico.**

Entorno	Ventajas	Desventajas
Wokwi (Simulación)	- No necesita componentes físicos- Gratuito- Simulación rápida- Ideal para principiantes- Permite visualizar el funcionamiento (LED, etc.)	- No simula fallos reales- Limitaciones de hardware- Menor precisión que el entorno real
Hardware Real	- Experiencia real- Alta precisión- Interacción física- Permite detectar errores reales (conexiones, voltaje, etc.)	- Tiene costo- Posibles daños en componentes- Mayor dificultad de uso- Requiere más tiempo para armar el circuito

8. Evaluación

No aplica para evaluacion

9. Bibliografía

1. Arduino Reference. Bitwise Operators. arduino.cc/reference
2. Mazidi, M. A. (2020). The 8051 Microcontroller. Pearson.
3. Barr, M. (2020). Programming Embedded Systems. O'Reilly.

- **Declaración del uso de IA**

  En la presente practica se utilizó la Inteligencia Artificia como herramienta de apoyo para comprender acerca de las operaciones bitwise, como está estructurado el código Arduino así mismo ayudo a comprender y optimizar mejor el código, además se usó para la redacción del informe, pero el desarrollo de esta actividad fue revisada por el autor.

10. Elaboración y Aprobación

Elaborado por	José Guamán **Docente**	Maria Chuico **Estudiante**
Aprobado por	Edison L Coronel Romero **Director de Carrera**	Firmado electrónicamente por: EDISON LEONARDO CORONEL ROMER Validar únicamente con FirmaEC